

B1 B 系列

環境建置與 你的第一張 UMAP

今天裝好環境，今天就跑出第一張圖。細節，後面十七集慢慢還。

約 30 分鐘 | 前置：A 系列（或等效基礎） | 資料：PBMC 3k

兩種學程式碼的方式

照抄網路教學

```
pbmc <- NormalizeData(pbmc)
pbmc <- RunPCA(pbmc)
pbmc <- FindClusters(pbmc,
  resolution = ???)
pbmc <- RunUMAP(pbmc, dims = ???)
```

**Error: 改一個參數就爆炸，
而且你不知道為什麼**

B 系列結束時

同一條流程，但每個參數
你都答得出「為什麼是這個值」

QC

標準化

PCA

分群

UMAP

B5

B6

B7

B8

B9



今天先把整條流程跑通一次

左：照抄，一改就爆。右：本系列的目標——每個參數你都答得出為什麼

本集只做一件事

今天裝好環境， 今天跑出你的第一張 **UMAP**。

每一步只講「做什麼」，「為什麼」標好路標，B5 到 B9 逐集講透。

這一集你會學到

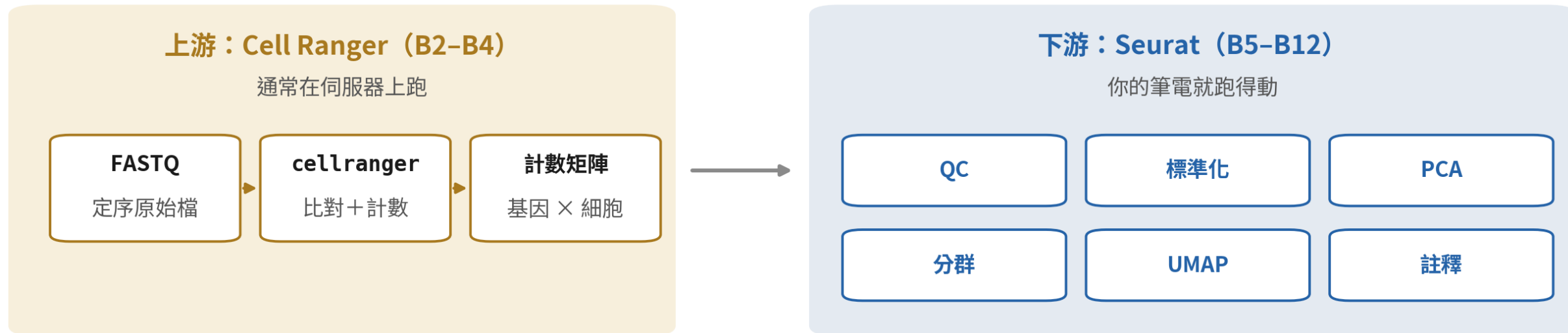
- 1 裝好 R、RStudio、Seurat v5，並驗證安裝成功
- 2 建立全系列共用的專案結構與 renv，說得出為什麼要鎖版本
- 3 用 15 行程式碼把 PBMC 3k 從讀檔跑到 UMAP，知道每行大概在做哪一步

01

全景與路線

先知道整條流程長什麼樣，再決定你要從哪裡走

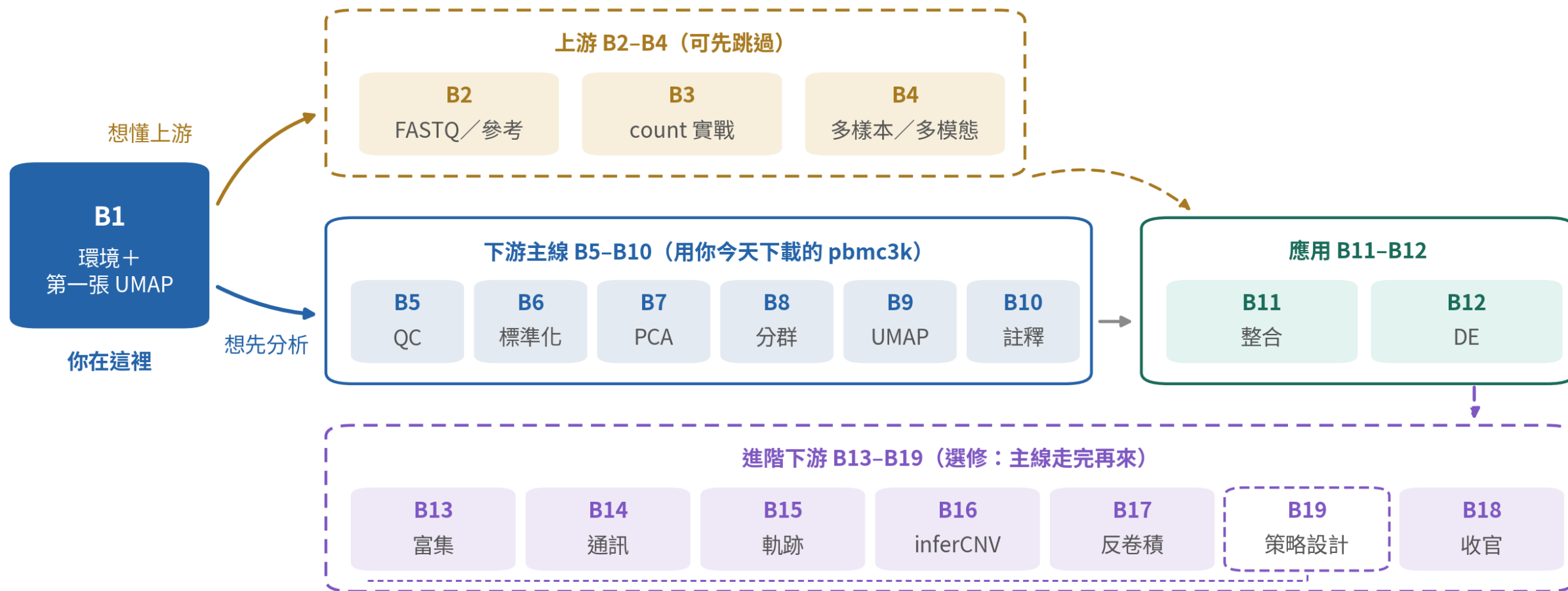
工具鏈全景：上游與下游



本集 quickstart：拿現成矩陣，直接走完下游一遍

上游把定序檔變成矩陣，下游把矩陣變成生物學——本集直接從矩陣起跑

十九集的地圖



兩條路都通：跳過 B2-B4 完全不影響 B5 起步；B13-B19 依研究問題選修（B19 是橫貫的策略總論），收官在 B18

B2-B4 講上游可先跳過；主線 B5-B12 是必走；B13-B19 進階下游選修（B19 是橫貫的策略總論）。下一站就是 B5

本系列的第一課

可重現性不是美德， 是論文的生存條件。

同一支腳本、同一份資料，兩年後要能跑出同一張圖。所以第一課是版本與 seed ，不是演算法。

02

安裝與驗證

R → RStudio → 平台工具 → Seurat v5 ，順序不能反

安裝順序



順序不能反：R 先裝好，RStudio 才找得到它；Rtools 缺了，很多套件編譯不過

四步各五分鐘；已經裝好的人，只需要跑最後的驗證

三個平台注意事項

Windows : Rtools



編譯套件的工具箱。
版本要對應 R 版本
(R 4.4 配
Rtools44)

Mac : 命令列 工具



第一次裝套件時跳出
安裝提示，同意它，
等它跑完

R 版本 $\geq$ 4.3



Seurat v5 需要。太
舊就整個重裝 R，不
要硬撐

裝 Seurat 與周邊

```
# CRAN 上的 Seurat 現在就是 v5.x  
install.packages("Seurat")
```

```
# 本集另外會用到的兩個  
install.packages(c("tidyverse", "renv"))
```

```
# Windows 若提示需要 Rtools：先裝 Rtools，重開 RStudio 再跑  
# Mac 若跳出 command line tools 安裝視窗：同意，等它裝完
```

第一次裝 Seurat 連相依套件要跑幾分鐘，出現紅字不一定是錯誤，看最後有沒有 error

驗證安裝

```
library(Seurat)  
packageVersion("Seurat")  
R.version.string
```

```
Loading required package: Seurat  
[1] '5.1.0'  
[1] "R version 4.4.1 (2024-06-14)"
```

你要看的是版本號開頭： Seurat 5.x、 R 4.3 以上，兩個都對才算裝好

第一節做完，你有了

一套能跑的 R 環境



R 4.3+ 、
RStudio 、平台工具
都就位

Seurat v5



版本號親眼驗證過，
不是「應該有裝」

一行驗證的習慣



`packageVersion()`
以後每次求救前先跑

03

專案結構與 `renv`

資料夾怎麼擺、版本怎麼鎖，全系列只講這一次

全系列共用的專案結構

scrna-course/

- └─ scrna-course.Rproj
- └─ renv.lock
- └─ data/
 - └─ download_data.R
 - └─ pbmc3k/
- └─ R/
 - └─ B01_first_umap.R
- └─ output/

專案根目錄

開專案一律點它

套件版本清單

原始資料：只進不改

下載腳本

解壓後的矩陣

分析腳本：一集一支

本集的 15 行

圖、rds、表格

路徑都是相對的

腳本裡只寫 data/...、output/...
整個資料夾搬去哪都能跑

資料與程式分開

data/ 裡的檔案永遠不動手改
所有處理都寫成 R/ 裡的腳本

產出可以隨時重生

output/ 整個刪掉也沒關係
重跑腳本就會長回來

data 放原始資料、R 放腳本、output 放產出；路徑一律相對於專案根目錄

建立專案與資料夾

```
# RStudio: File → New Project → New Directory → "scma-course"
```

```
# 之後每次都從 scma-course.Rproj 開專案
```

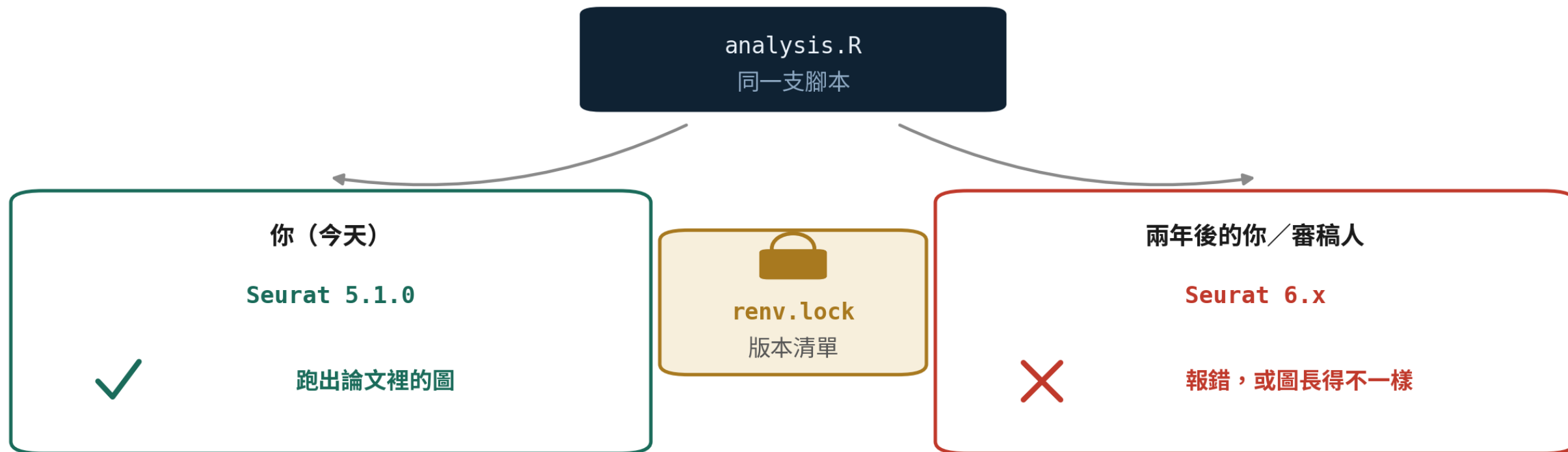
```
dir.create("data") # 原始資料：只進不改
```

```
dir.create("R") # 分析腳本：一集一支
```

```
dir.create("output") # 圖、rds、表格：可隨時重生
```

開了 .Rproj，工作目錄自動是專案根——這就是全系列不用 `setwd` 的原因

為什麼要鎖版本



`renv` 把「這個專案用了哪些套件、什麼版本」寫進 `renv.lock`；換電腦 `renv::restore()` 就能還原同一套環境

套件更新會改變預設值、改物件結構，甚至改結果—— `renv.lock` 是你的保險

renv : 兩個指令就上保險

```
renv::init() # 給這個專案一套獨立的套件庫
```

... .. 套件都裝齊之後，任何時候：

```
renv::snapshot() # 把目前的套件版本清單寫進 renv.lock
```

- Lockfile written to "~ /scma-course/renv.lock".

renv.lock 跟著專案走；別台電腦 `renv::restore()` 就能還原同一套環境

下載 PBMC 3k : data/download_data.R

```
url<-paste0("https://cf.10xgenomics.com/samples/cell/",  
            "pbmc3k/pbmc3k_filtered_gene_bc_matrices.tar.gz")  
options(timeout= 600)      # 網速慢時不要中途放棄  
download.file(url, "data/pbmc3k.tar.gz", mode = "wb")  
untar("data/pbmc3k.tar.gz", exdir = "data/pbmc3k")  
list.files("data/pbmc3k", recursive = TRUE)
```

```
[1] "filtered_gene_bc_matrices/hg19/barcodes.tsv"  
[2] "filtered_gene_bc_matrices/hg19/genes.tsv"  
[3] "filtered_gene_bc_matrices/hg19/matrix.mtx"
```

約 7.6 MB ; 你要看的是最後這三行——三個檔案都在，才算下載成功

下載下來的是什麼



Read10X 吃的是裝著這三個檔案的「資料夾」，它自己把三者組回一個矩陣（細節 B3 會講）

10x 官方的 filtered 矩陣：2,700 顆 PBMC，筆電輕鬆跑

第二節做完，你有了

一個標準專案



data/ R/ output/
加 .Rproj，全系列
都用它

renv.lock



環境有了快照，兩年
後也能還原

pbmc3k 資料



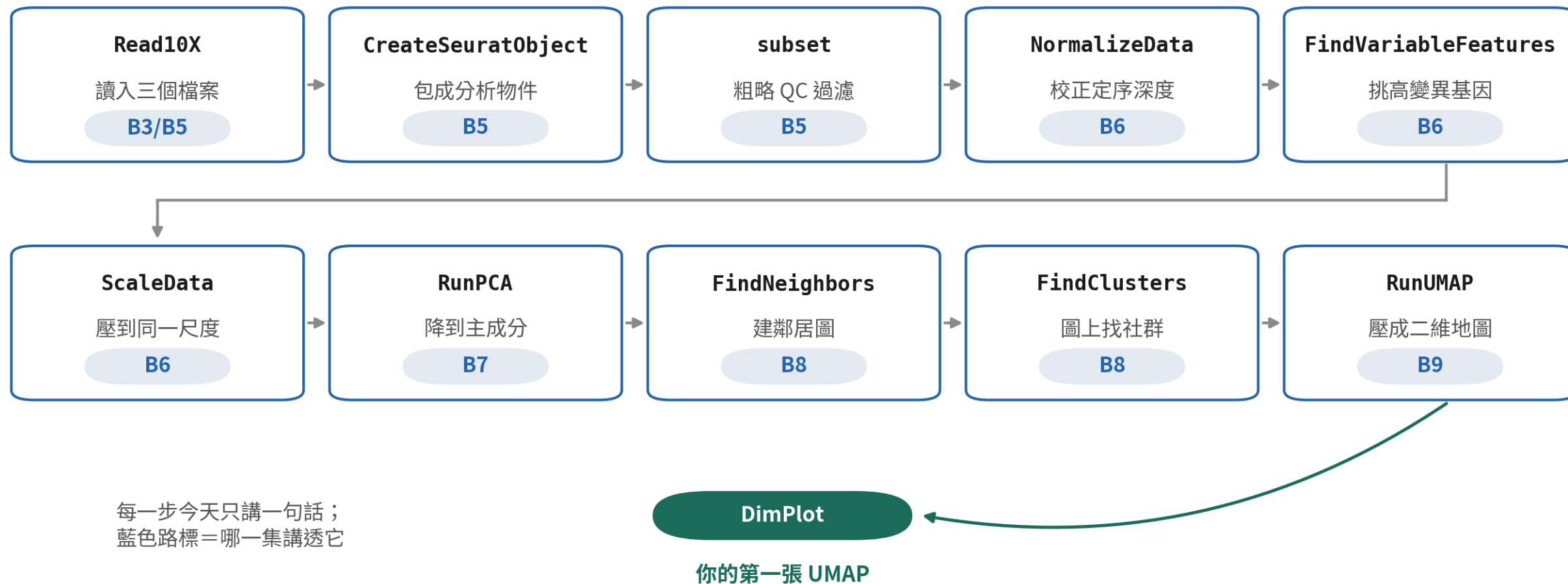
三個檔案就位，B5
到 B10 都用這一份

04

15 行，跑到第一張 UMAP

每一步一句話；「為什麼是這些值」的路標都已標好

整條流程，一張圖



每一步今天只講一句話；
藍色路標＝哪一集講透它

十個函式一路串到 DimPlot；藍色路標是「哪一集講透它」

第一段：讀檔、建物件、粗略 QC

```
library(Seurat)
set.seed(1234) # 全系列固定的隨機種子

pbm c.data <- Read10X(data.dir =
  "data/pbm c3k/filtered_gene_bc_matrices/hg19/") # → B3/B5
pbm c <- CreateSeuratObject(counts = pbm c.data,
  project = "pbm c3k",
  min.cells = 3, min.features = 200)
pbm c[["percent.m t"]] <- PercentageFeatureSet(pbm c,
  pattern = "^M T-")
pbm c <- subset(pbm c, subset = nFeature_RNA > 200 &
  nFeature_RNA < 2500 & percent.m t < 5) # → B5
pbm c
```

An object of class Seurat
13714 features across 2638 samples within 1 assay

2,700 → 2,638：粗略 QC 濾掉 62 顆。這些閾值今天先照抄，B5 教你自己定

第二段：標準化到 PCA

```
pbm c <- NormalizeData(pbm c)      # 校正定序深度    → B6  
pbm c <- FindVariableFeatures(pbm c) # 挑 2000 個 HVG → B6  
pbm c <- ScaleData(pbm c)          # 壓到同一尺度    → B6  
pbm c <- RunPCA(pbm c)              # 降到主成分空間  → B7
```

四行都用預設值。每個預設值背後都有一個假設—— B6、B7 逐一拆給你看

第三段：分群與 UMAP

```
pbm c <- FindNeighbors(pbm c, dim s = 1:10) # 建鄰居圖 → B8  
pbm c <- FindClusters(pbm c, resolution = 0.5) # 找社群 → B8  
pbm c <- RunUMAP(pbm c, dim s = 1:10) # 壓成二維 → B9
```

```
Number of nodes: 2638  
Number of edges: 95965  
Running Louvain algorithm ...  
Maximum modularity in 10 random starts: 0.8723  
Number of communities: 9
```

你要看的是最後一行：9 個 communities，就是 9 群

第四段：畫圖、存檔

```
D in Plot(pbm c, reduction = "um ap", label= TRUE)

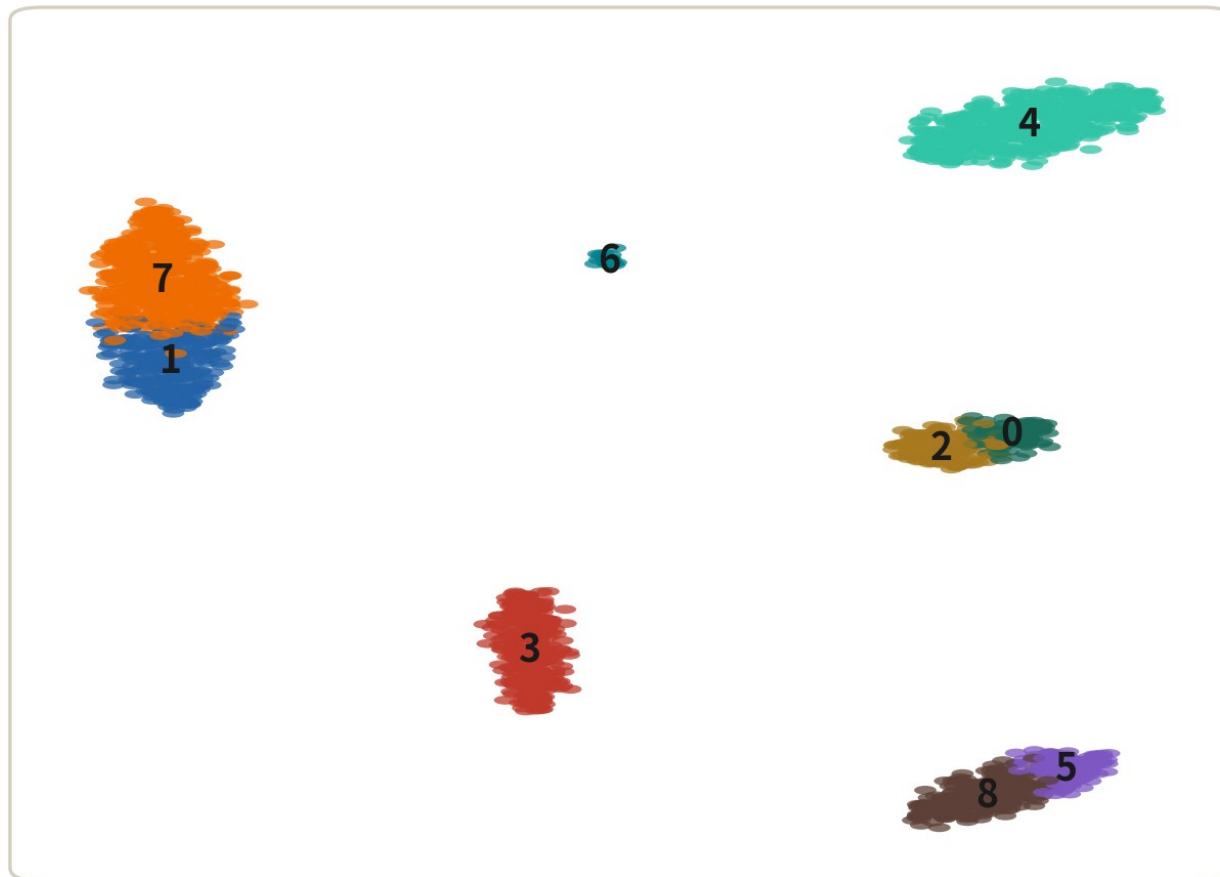
ggplot2::ggsave("output/B01_first_um ap.png",
                width = 7, height = 5.5, dpi = 200)
saveRDS(pbm c, "output/pbm c_b01_first_run.rds")
sessionInfo() # 版本紀錄，貼進你的分析筆記
```

物件存成 rds、圖存進 output、sessionInfo 記版本——收工三件套，每集片尾都一樣

你的第一張 UMAP

2,638 顆細胞
9 個 cluster

一顆點＝一顆細胞
顏色＝分到的群



這些群是誰？
T 細胞？B 細胞？
→ B10 教你回答

每個參數
為什麼是這個值？
→ B5-B9

模擬資料（錄影時換 pbmc3k 實跑）

2,638 顆細胞、9 群。恭喜——你已經把整條下游流程跑通了一次

這張圖的正確心態

先跑通，再跑懂。
今天的每個參數都是預設值。

跑通給你信心與全景；跑懂是 B5 到 B9 的工作。兩個都要，順序是這個。

A 系列術語快速回顧

barcode

細胞條碼：油滴的身分證，矩陣的一欄

UMI

分子流水號：數分子，不數 reads

count matrix

基因 × 細胞計數表，一切的起點

doublet

兩顆細胞擠進一顆油滴，假裝一顆

cluster

表現量相似的細胞群（演算法分的）

UMAP

高維壓成二維的地圖，只供看不供量

六個詞出自 A 系列；後面各集直接使用，不再重教

這六個詞後面各集直接用，不再重教；忘了就回 A 系列對應集

05

踩雷區

兩個雷都跟「環境」有關，也都實際跑給你看

雷一：不管版本，抄到什麼跑什麼

雷是什麼

網路教學照抄，沒注意它是 Seurat v4 時代寫的。



為什麼會踩

教學不會過期標示，Google 排前面的常常是舊文章。

踩了會怎樣

v5 物件沒有 @counts slot，直接報錯；更陰險的是「不報錯但行為變了」的情況。

同一行 code , v4 過、v5 爆

網路上的 v4 教學碼

```
counts <- pbmc@assays$RNA@counts
```

v4 : 矩陣放在 @counts slot

在 v5 執行：

**Error: no slot of name "counts" for
this object of class "Assay5"**

Seurat v5 的寫法

```
counts <- LayerData(pbmc,  
                    layer = "counts")
```

v5 : 矩陣住在 layers 裡

counts

data

scale.data

layers : 原始、標準化、縮放三層並存

網路教學先看發文日期：2023 年以前的多半是 v4 語法，照抄前要翻譯

v5 把矩陣搬進 layers；讀舊教學前，先看它的發文日期

雷二： `setwd()` 寫死絕對路徑

雷是什麼

腳本第一行 `setwd("C:/Users/ 自己的名字 /...")`。



為什麼會踩

當下最直覺——路徑錯了就把它寫死，眼前的錯誤消失。

踩了會怎樣

換電腦、給同學、兩年後的自己，第一行就爆炸；中文使用者名稱的路徑還會讓部分套件讀檔失敗。

絕對路徑 vs RStudio Project

寫死絕對路徑

```
setwd("C:/Users/ming/Desktop/  
analysis_final_v2")
```

換一台電腦、換一個人：

```
Error in setwd(...) :  
cannot change working directory
```

RStudio Project + 相對路徑

```
# 開 .Rproj，工作目錄自動=專案根  
Read10X("data/pbmc3k/...")
```

整個資料夾搬到任何電腦：



照樣能跑，一個字都不用改

鐵則：腳本裡永遠不出現 `setwd()` 與絕對路徑——這也是本系列所有腳本的寫法

專案 + 相對路徑：整個資料夾搬到哪，腳本一個字都不用改

其他常見安裝雷，快查表

R 版本太舊



裝 Seurat 時報「需要 R $\geq$ 4.x」→
重裝新版 R，別找舊版套件

中文路徑



使用者名稱或資料夾是中文 → 專案放到
純英文路徑，例如 C:/scrna

記憶體不足



pbmc3k 只要 8GB；大資料集才需要
加 RAM 或上伺服器

GitHub 套件裝不動



先裝 remotes；公司或校園網路擋
GitHub 時換網路再試

踩雷區小結

**環境問題不是你的程度問題，
是每個人的第一關。**

版本驗證過、路徑走相對、renv 上了鎖——這三件做了，九成的「玄學錯誤」不會發生。

⑤ +

複雜樣本策略室

標準流程走完了——現在想像你手上不是 PBMC，是一顆腫瘤

雙軌設計：走讀＋策略室

複雜樣本＝腫瘤、發炎組織、多區域取樣、血液腫瘤；策略＝決策，不是新流程

主線走讀：一字不動



pbmc/ifnb 標準流程照舊——先把每一步跑通、跑懂

策略室：同一步，不同決策



每集片尾 2-5 頁，不重教工具，只講換成複雜樣本時決策哪裡不同、為什麼

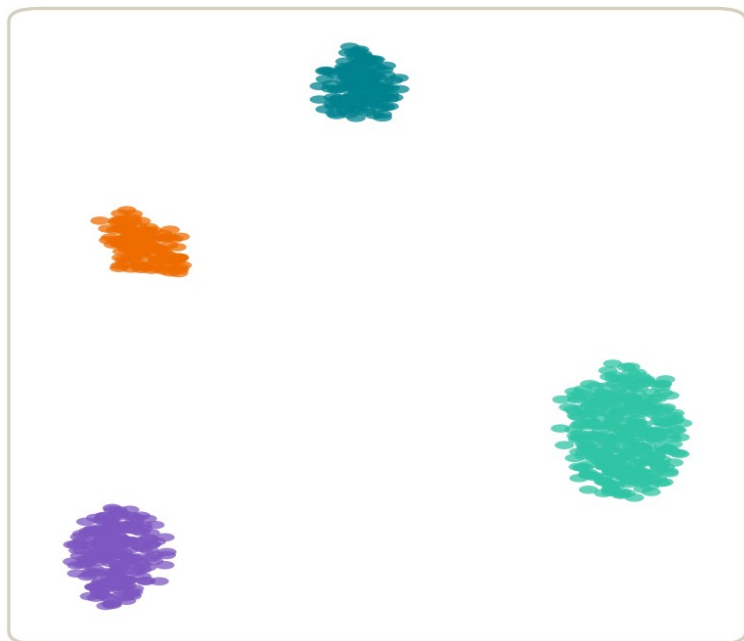
B19：不同腫瘤，不同戰法



各集累積的策略，最後總裝成一整集的策略設計課

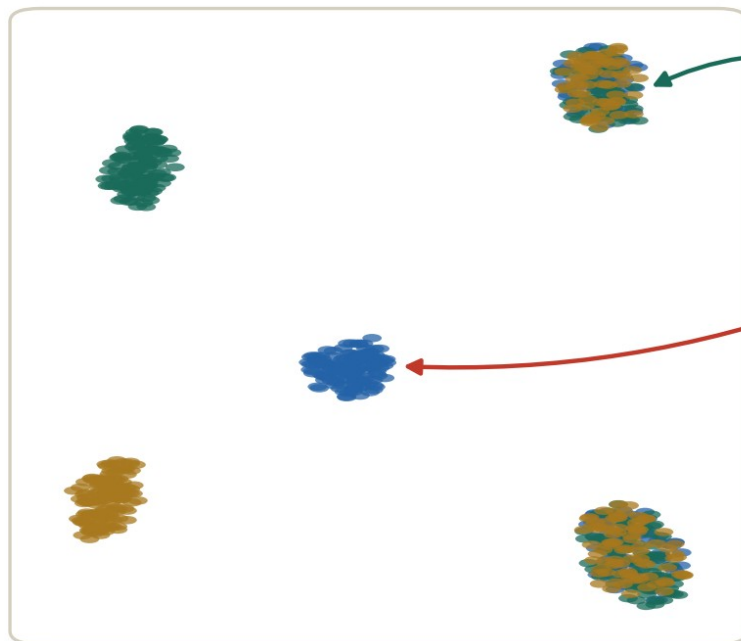
同一條 15 行，餵一顆腫瘤

PBMC : quickstart 的世界



顏色 = 細胞型別；一群 ≈ 一種型別

腫瘤：同一條 15 行



● 病人 1 ● 病人 2 ● 病人 3

免疫／基質細胞：
三個病人混在一起——正常

惡性細胞：
一位病人一座島

島裡還可能混著正常細胞
marker 分不出 → 靠 CNV (B16)

程式照樣跑得完、圖照樣漂亮——但「群」的意義變了；這三座暗礁，每集片尾的策略室逐一拆

模擬資料

程式照樣跑完、圖照樣漂亮——但惡性細胞按病人成島、惡性與正常可能混群、病人才是統計單位

本集 checklist



R 4.3 以上與 RStudio 都裝好、開得起來



`packageVersion("Seurat")` 回報 5.x



專案建好：data/ R/ output/ 加 renv.lock



pbmc3k 三個檔案躺在 data/pbmc3k/ 裡



R/B01_first_umap.R 從頭到尾零錯誤



output/ 裡有你的第一張 UMAP 與 .rds

六格全勾，你就準備好進 B5（或 B2）了

一句話總結

**環境是實驗設備的一部分：
版本要驗、路徑要相對、`seed` 要固定。**

今天那張 UMAP 的每個參數都是預設值——這句話 B5 見。

自測 1

為什麼本系列第一課就要求 **renv** 鎖版本？

- A `renv` 會讓程式跑得比較快
- B 套件更新可能改變預設值與結果，鎖版本才能重現分析
- C 不裝 `renv` 就不能裝 `Seurat`
- D `renv` 可以自動修正程式錯誤

答 B。同一支腳本在不同版本下可能報錯、甚至跑出不同結果。`renv.lock` 記下每個套件的版本，讓兩年後的你、你的同學、審稿人都能還原同一套環境。

自測 2

Read10X() 的參數該指向什麼？

- A `matrix.mtx` 這個檔案
- B 下載回來的 `tar.gz` 壓縮檔
- C 裝著三個檔案（`barcodes`、`genes`、`matrix`）的資料夾
- D 10x 官網的下載網址

答 C。Read10X 吃的是資料夾路徑，它自己把 `barcodes.tsv`、`genes.tsv`、`matrix.mtx` 組回一個稀疏矩陣。指到單一檔案或壓縮檔都會報錯——這是新手最常見的第一個錯誤訊息。

自測 3

同學傳來的腳本第一行是

setwd("C:/Users/ming/Desktop/analysis")，在你電腦上跑不動。
正確的修法是？

- A 把路徑改成你自己電腦的絕對路徑
- B 建 RStudio Project，刪掉 setwd，改用相對路徑
- C 把資料搬到跟他一樣的路徑
- D 用管理員權限執行 RStudio

答 B。改成自己的絕對路徑（A）只是把同一顆雷傳給下一個人。正解是專案 + 相對路徑：開 .Rproj 之後工作目錄自動是專案根，data/、output/ 在任何電腦都成立。

下集預告

B2：參考基因組與 FASTQ (想先分析？直接跳 B5)

今天那張 UMAP，每一步都用了預設值——預設值是對的嗎？誰決定的？

上游派往 B2 看矩陣是怎麼來的；分析派直接去 B5，從 QC 的第一條線開始算帳。